

VQ-VAE and PixelCNN for HipMRI Prostate Cancer Image Generation

Author: Snehin Raj Singh Kukreja (48397690)

1. Overview

Purpose & Problem

This project tackles generative modelling of 2D prostate MRI slices from the HipMRI Study on Prostate Cancer dataset [1].

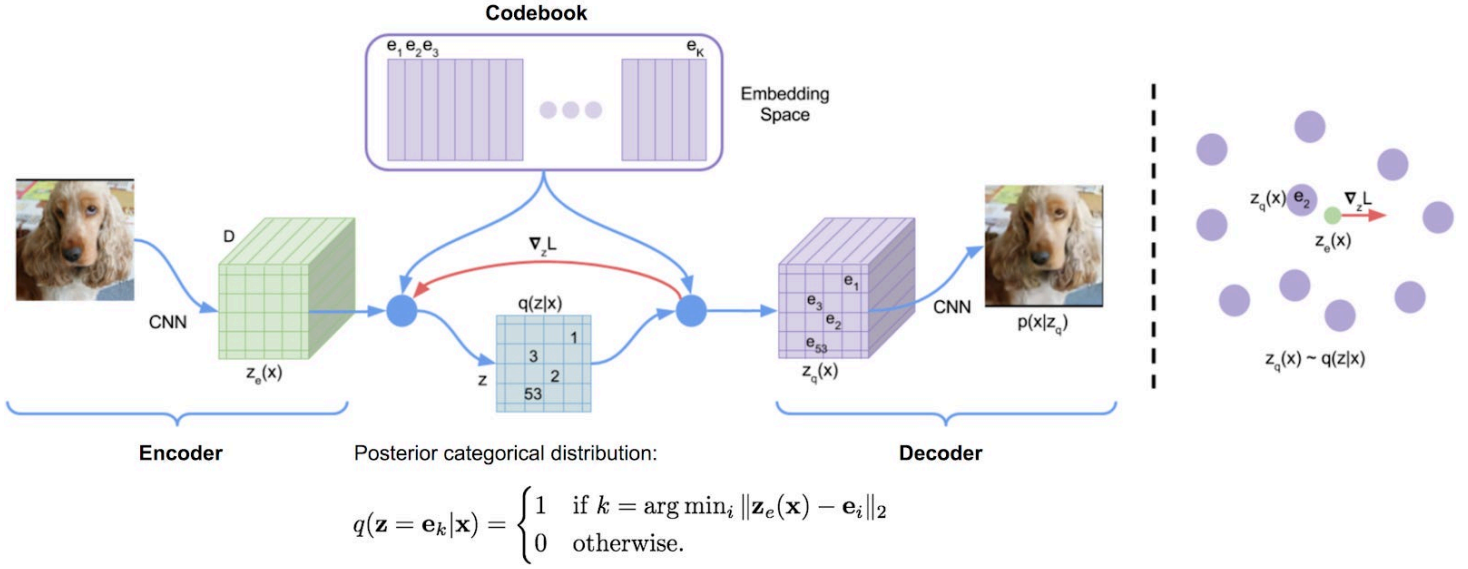
The aim is to produce realistic, structurally faithful MRI images that can support research, radiotherapy planning, and data augmentation. Following the COMP3710 “**Hard**” project brief, the model must generate “reasonably clear images” with a Structural Similarity Index Measure (SSIM) ≥ 0.60 .

Generative modelling in medical imaging is a challenging area due to the need for both anatomical accuracy and training stability of deep learning models. This project explores how modern discrete latent models can achieve this balance by learning interpretable representations of visual structures.

How it Works

The model is a two-stage pipeline designed to first learn a "vocabulary" of visual features and then learn the "grammar" of how to arrange them.

Stage 1: VQ-VAE



(Diagram sourced from Analytics Vidhya [2])

The first stage learns a discrete latent representation of the HipMRI slices. It consists of an encoder, vector quantizer, and decoder. The encoder compresses each image, the quantizer replaces latent vectors with the nearest entries from a learnable codebook, and the decoder reconstructs the input.

Training minimises the Loss function $\mathcal{L}_{\text{total}}$:

$$\mathcal{L}_{\text{total}} = \underbrace{\|x - \hat{x}\|_2^2}_{\text{reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{codebook}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{commitment}}$$

where

- x is the input image,
- $\hat{x}$ is the reconstructed output,
- $z_e(x)$ is the encoder output,
- e is the nearest embedding vector from the codebook,
- $\text{sg}[\cdot]$ denotes the stop-gradient operator (no gradient passed), and
- β is the commitment cost, set to 0.25 in this project.

The total loss balances accurate reconstruction with stable and efficient codebook usage.

- **Reconstruction loss** using Mean Squared Error (MSE) between original and reconstructed images.
- **Vector-quantisation loss** that aligns encoder outputs with the codebook.

- **Commitment loss** weighted by the `commitment_cost` parameter (0.25) to encourage consistent usage of the codes in the codeblock.

Hyperparameters and justification

Data and normalization

- `IMAGE_SIZE = 128` : downsizes variable slice sizes to a square for simpler batching and faster training; keeps enough structure for pelvic anatomy.
- `IN_CHANNELS = 1` : MRI slices are single-channel.
- Normalization to `[-1, 1]` : matches decoder tanh-style output scaling and sets SSIM `data_range=2.0` .

Optimizer and schedule

- I used Adam because it combines the benefits of momentum (from SGD with momentum) and adaptive learning rates (from RMSProp).
For VQ-VAE training, the encoder–decoder and quantizer updates have gradients with very different magnitudes. Adam’s per-parameter learning rate adjustment keeps updates balanced without manual tuning.
- `LEARNING_RATE = 1e-4` was a common stable default for Adam with reconstruction losses; higher values caused the model to collapse, lower values slowed convergence.
- `WARMUP_EPOCHS = 3` : This gradual ramp-up reduces early quantizer instability in VQ-VAE.

Batching and runtime

- `BATCH_SIZE = 64` This balances gradient estimate quality with GPU memory at 128×128. Since I trained the model locally too on a RTX4060 Laptop with 8GB of VRAM.
- `NUM_WORKERS = 8 (cluster) / 0 (local)` : parallel I/O on HPC; avoids Windows multiprocessing issues locally.
- `EPOCHS = 70` : There was a plateau in validation SSIM observed before 70 as shown in the loss curves later on. Further epochs gave diminishing returns.

VQ-VAE architecture

- `HIDDEN_CHANNELS = 128` — capacity appropriate for 128×128 inputs without exhausting memory.
- `NUM_RES_BLOCKS = 2`, `RES_CHANNELS = 64` — residual depth adds nonlinearity while keeping runtime manageable.
- Reconstruction loss = MSE provided stable feedback to the model for reconstruction; aligns with SSIM improvements as shown in the loss curves.

Vector quantizer

- `NUM_EMBEDDINGS = 512` had sufficient codebook diversity without codebook collapse.
- `EMBEDDING_DIM = 128` matched encoder channel width, reducing projection overhead and preserving detail.
- `COMMITMENT_COST = 0.25` — standard setting from VQ-VAE to balance codebook usage and encoder drift.

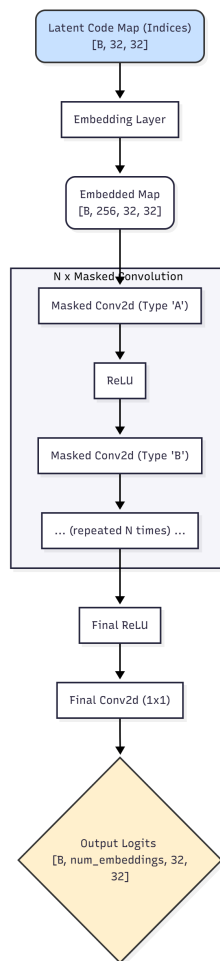
Model selection and metrics

- `Validation metric = SSIM (data_range = 2.0)` — measures structure preservation; `data_range` matches `[-1, 1]` scaling.
- `EARLY_STOP_SSIM = None` to allow for the training of the model to saturate and hit a maximum validation SSIM score.

Stage 2: Learning the Spatial Structure (PixelCNN)

While the VQ-VAE learns *what* to draw, it doesn't learn *how* to arrange the features coherently. This is the job of the **PixelCNN**, which acts as a powerful **prior** over the discrete latent space.

1. **Training:** After the VQ-VAE is trained and frozen, its encoder is used to convert the entire training dataset into a set of discrete latent maps (grids of codebook indices). The PixelCNN is then trained on these maps.
2. **Autoregression:** The PixelCNN learns to predict the next code index in a grid based on all the previous indices "above and to the left" of it. It learns the statistical patterns and spatial relationships of the visual vocabulary.



During the final generation step, the trained PixelCNN creates a completely new, structured latent map from scratch, one "pixel" (code index) at a time. This synthetic map is then passed to the VQ-VAE's decoder to produce a novel, high-quality image that respects the learned spatial patterns of the original dataset.

PixelCNN prior and justification

- Target = code indices from VQ-VAE encoder which trains a prior over the discrete latent grid.
- Loss = cross-entropy since it's a classification task over `NUM_EMBEDDINGS = 512` categories per latent position.
- `Hidden_dim = 256`, `num_layers = 7` best results for a PixelCNN width/depth for 32×32 latent maps (128/4).
- `Optimizer = Adam`, `Learning Rate LR = 1e-4` Adam combines the benefits of momentum (from SGD with momentum) and adaptive learning rates (from RMSProp). Adam's per-parameter learning rate adjustment keeps updates balanced without manual tuning. Higher learning rates caused the model to collapse during training.
- `Scheduler = cosine annealing` for a smooth decay of the learning rate which helps prevent collapse during training.
- `Epochs = 50` : The likelihood plateaus before 50; longer runs add little visual improvement.

2. Data and Preprocessing

Dataset

The model is trained on the HipMRI Study on Prostate Cancer dataset [1], which consists of pre-processed 2D slices stored as NIfTI files (`.nii.gz`). The dataset is divided into training, validation, and testing sets.

Pre-processing

The following pre-processing steps are applied in `dataset.py` :

1. **Resizing:** All images are resized to a uniform dimension of **128x128 pixels** to keep consistent input for the model.
2. **Normalization:** Pixel values are normalized to the range `[-1, 1]` . This is a standard practice that helps stabilize training and aids the model's convergence.

(Reference: The normalization method is a common technique in deep learning for image data.)

Data Splits Justification

The dataset is pre-split into `train` , `validate` , and `test` directories. This project respects this split. The **training set** is used exclusively to train the model parameters. The **validation set** is used during training to make key decisions, such as when to save the best model and when to reduce the learning rate. Finally, the **test set** is held out and used only once at the very end in `predict.py` to provide an unbiased, final evaluation of the model's performance on completely unseen data.

3. Project Structure

The repository follows a modular design separating configuration, data loading, model definition, training, and evaluation to an easy to maintain and configure system.

```
/
├── diagrams/           # Folder for storing diagrams like vqvae_diagram.png
├── checkpoints/        # Directory for saved model weights
├── predictions/        # Directory for final output images
├── main.py             # Main entry point for all operations
├── config.py           # Centralized configuration for all hyperparameters
├── modules.py          # VQ-VAE and PixelCNN model architectures
├── dataset.py          # Data loading and preprocessing pipeline
├── train.py            # The VQ-VAE training and validation loop
├── train_pixelcnn.py   # The PixelCNN training loop
├── predict.py          # Script to load models and generate final results
├── requirements.txt     # Library dependencies for reproducibility
└── README.md           # This file
```

4. Dependencies and Reproducibility

Environment Setup

The project uses a Conda-managed environment for full reproducibility.

All dependencies, including Python version and library specifications, are defined in the `environment.yml` file.

To recreate the environment:

```
conda env create -f environment.yml
conda activate vqvae_env
```

The environment can be exported from an existing setup using:

```
conda env export --no-builds > environment.yml
```

Reproducibility Notes

- All hyperparameters and training settings are defined in `config.py`, which controls both the VQ-VAE and PixelCNN stages.
- The same configuration file can be used on local machines or the Rangpur HPC cluster for consistent experiments.

- The Conda environment defined in `environment.yml` ensures version-controlled dependencies across systems.
- Model checkpoints, loss plots, and SSIM metrics are automatically saved in the `checkpoints/` directory for reproducibility and review.
- The SSIM metric uses `data_range=2.0` to match the normalized image range of $[-1, 1]$.

5. Usage Instructions

The project is run from the terminal using `main.py`, which supports three modes:

`train`, `train_pixelcnn`, and `predict`.

Running on Rangpur (HPC)

On Rangpur, no additional arguments are required because the dataset path and worker settings are already configured for the cluster.

```
# Train the VQ-VAE model
```

```
python main.py train
```

```
# Train the PixelCNN prior
```

```
python main.py train_pixelcnn
```

```
# Generate reconstructed and new images
```

```
python main.py predict
```

Running on a Local Machine

When running locally, add the `--local` flag to use the local dataset path. E.g.

```
# Train the VQ-VAE model locally
```

```
python main.py train --local
```

```
# Train the PixelCNN prior locally
```

```
python main.py train_pixelcnn --local
```

```
# Generate reconstructed and new images locally
```

```
python main.py predict --local
```

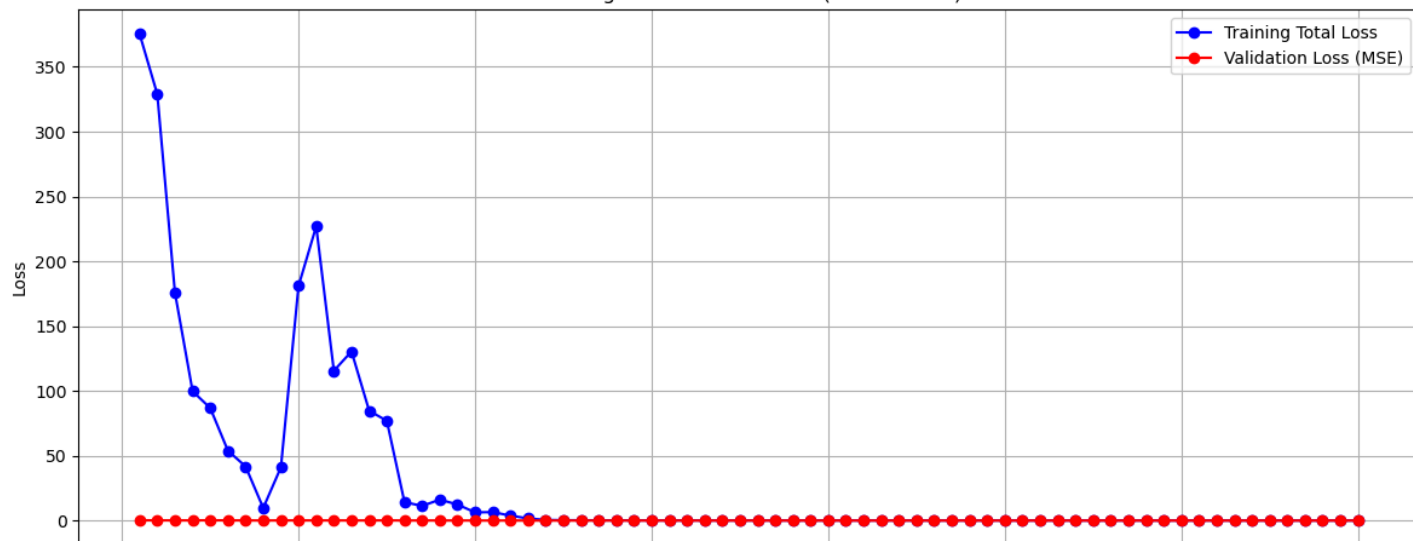

All outputs (plots, checkpoints, and generated images) are saved automatically to the `checkpoints/` and `predictions/` directories

6. Results and Analysis

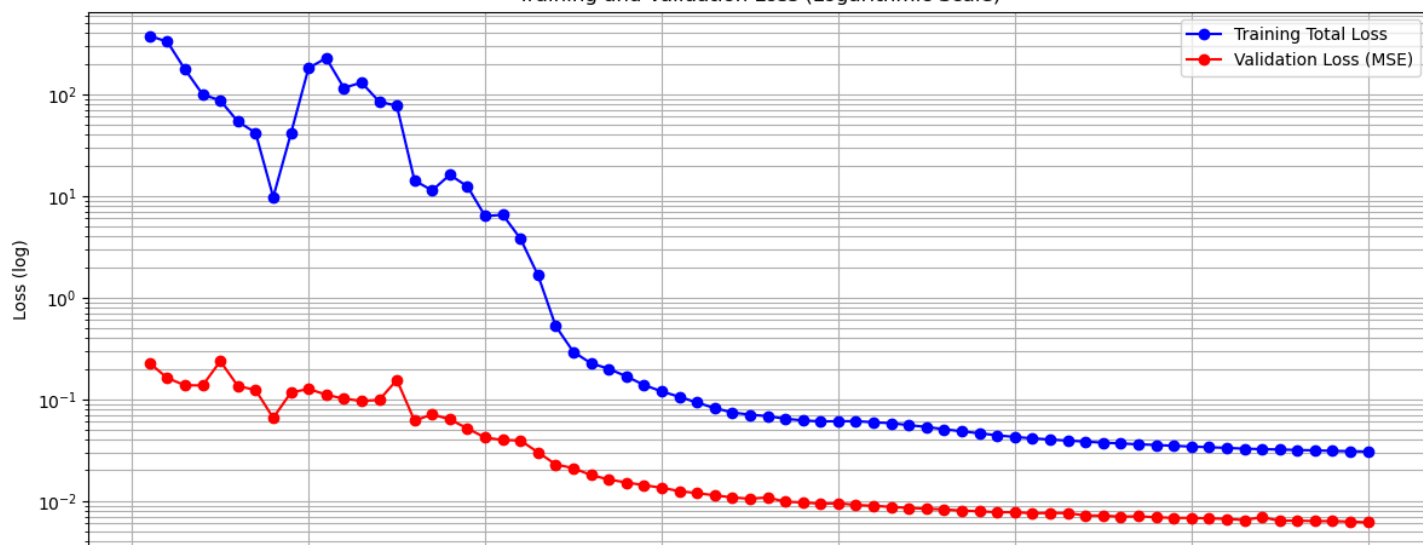
Training Progress

The VQ-VAE model was trained until the early stopping condition was met, indicating that it reached the target performance efficiently. The plot below shows the training loss, validation loss, and validation SSIM over the course of training. There is a big spike in loss at 10 epochs followed by a steady decrease and corresponding increase in SSIM. This shows that the model initially encountered an unstable loss landscape and quickly recovered. The validation SSIM score steadily increased till ~0.85 and plateaued for the last few epochs.

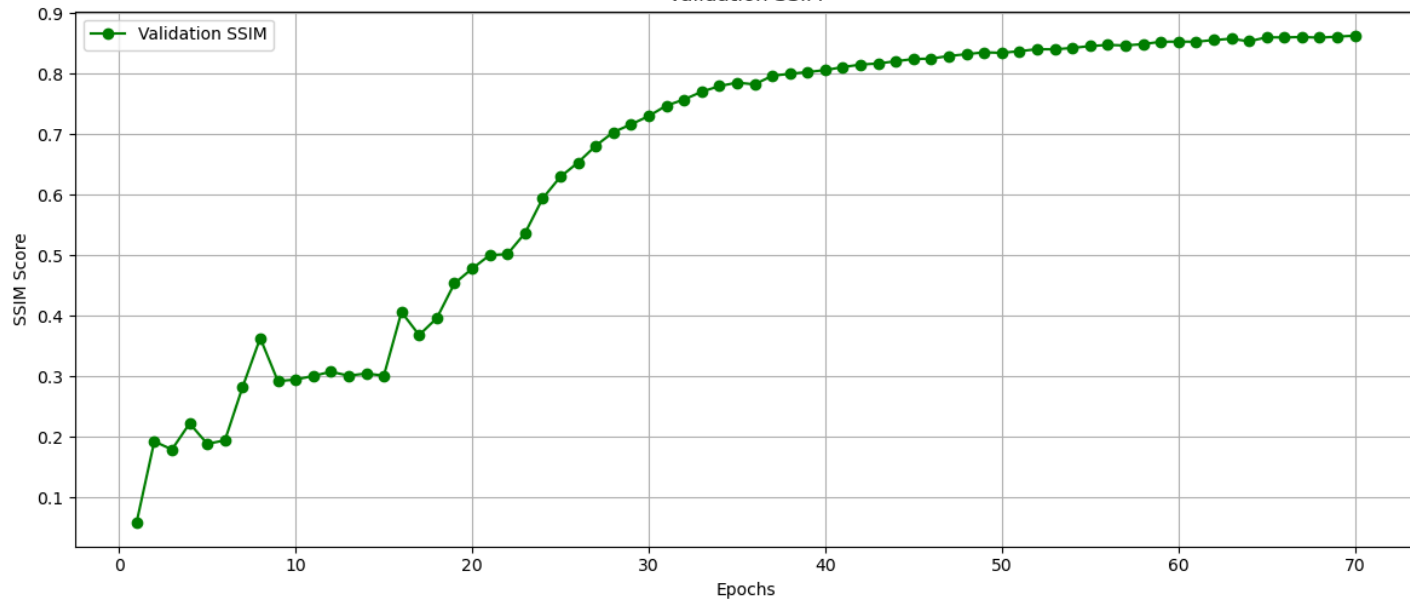
Training and Validation Loss (Linear Scale)



Training and Validation Loss (Logarithmic Scale)



Validation SSIM



Final Performance on Test Set

The fully trained model was evaluated on the unseen test set to provide a final, unbiased measure of its performance.

Overall Test Set SSIM: 0.8762

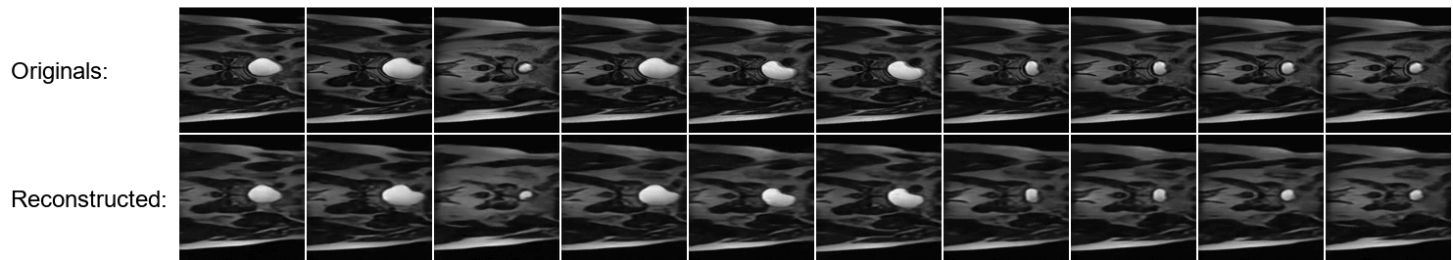
This result surpasses the project's target of 0.60, confirming the model's strong ability to accurately reconstruct high-fidelity images as required.

Qualitative Analysis: Best and Worst Cases

To gain deeper insight into the model's behavior, the `predict.py` script automatically identifies and saves the 10 best and 10 worst reconstructions from the test set based on their individual SSIM scores.

Top 10 Best Reconstructions (by SSIM)

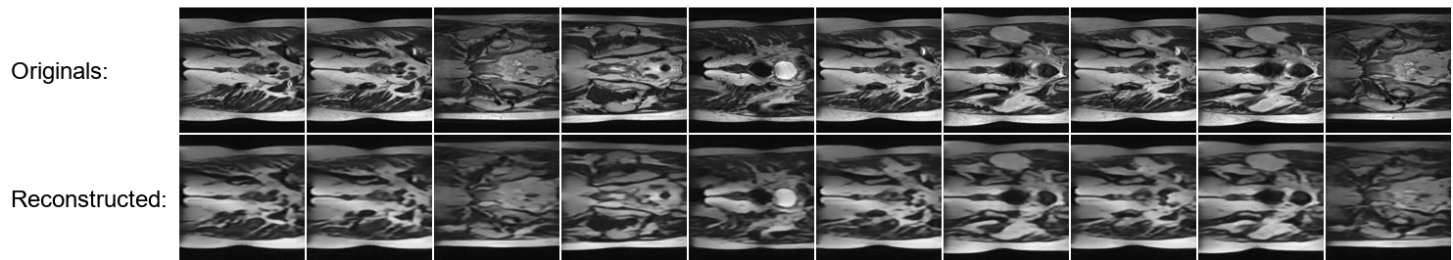
case_040



The best-case reconstructions are nearly indistinguishable from the originals, showing the model's success in capturing key anatomical structures and textures.

Top 10 Worst Reconstructions (by SSIM)

case_040



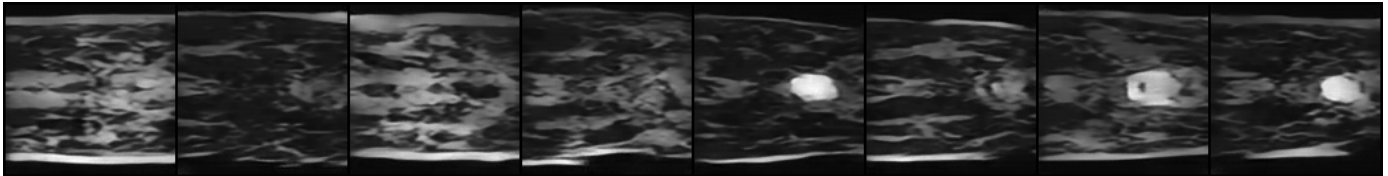
The worst-case images, are still structurally coherent, but contain for noise and struggle to capture finer grained complexities in the original images

Interestingly the best images and worst images seem to be grouped as similar type of images. This suggests that the model is better at reconstructing certain type of MRI images over others depending

on the label.

Final Generated Images

The final PixelCNN-generated samples (shown below) shows some of the broad structures similar to the HipMRI data but lack finer anatomical detail like in the originals above. This indicates that the model learned the overall distribution of tissue textures and contrasts but not the high-frequency boundaries or organ edges present in real slices.



This limitation likely arises from the small latent grid (32×32 for 128×128 inputs) and the discrete codebook bottleneck, which compresses fine spatial information.

Possible improvements include using a deeper PixelCNN, increasing the embedding dimension, or adopting a hierarchical approach such as VQ-VAE-2 to capture multi-scale features.

7. Future Work

Several extensions could improve both reconstruction quality and generative fidelity:

7.1. Hierarchical VQ-VAE (VQ-VAE-2)

Introduce a multi-scale latent hierarchy to capture global and local structures separately, allowing finer detail to be generated better.

7.2. Conditional Generation

Add conditioning variables such as slice index or patient label to guide the PixelCNN, producing more coherent and anatomically consistent outputs.

7.3. Dataset Augmentation and Balancing

Expand the dataset or apply contrast and orientation augmentations to improve codebook diversity and generalization ability of the model.

8. Declaration of AI Assistance

The Python source files for this project (`config.py` , `dataset.py` , `modules.py` , `train.py` , `train_pixelcnn.py` , `predict.py` , and `main.py`) were developed with assistance from **Google Gemini 2.5 Pro** [3].

The model was used to generate and refine code structure and documentation. See each file for more information.

9. References

[1] Australian e-Health Research Centre, CSIRO. *HipMRI Study on Prostate Cancer (open dataset)*. DOI: [10.25919/45t8-p065](https://doi.org/10.25919/45t8-p065)

[2] Koo, J. (2021). *An Overview on VQ-VAE : Learning Discrete Representation Space*. Analytics Vidhya. <https://medium.com/analytics-vidhya/an-overview-on-vq-vae-learning-discrete-representation-space-8b7e56cc6337>

[3] Google DeepMind. *Gemini 2.5 Pro* [Large Language Model]. Available at: <https://deepmind.google/technologies/gemini/>